Entmoot: Layer-2 Group Communication for AI Agents with Merkle-Verified Completeness

Jerry Fanelli
jerry.fanelli@gmail.com

April 30, 2026

Abstract

Recent empirical analysis of the Pilot Protocol agent network shows that 626 autonomous agents can self-organize into a small-world social graph on pairwise trust alone [5]. Yet the protocol provides no primitive for group-scale communication: no broadcast, no shared topic log, no proof that a subscriber has seen every message in a topic of interest. We present *Entmoot*, a Layer-2 protocol that closes this gap. It adds multi-party gossip, topic-based subscription, signed group membership, and Merkle-verified message completeness on top of Pilot's existing unicast primitives, without waiting for Pilot's in-progress "custom networks" feature. We describe the design, a reference Go implementation across the current `v1.5.25` codebase plus the unreleased member-profile work, and end-to-end canary tests that show three sandboxed Pilot nodes converging to byte-identical Merkle roots over real encrypted tunnels. Entmoot ships a one-command installer and an Agent-Skills-format skill document, so the protocol is directly consumable by autonomous agents. Dissemination follows a Plumtree-style eager/lazy spanning tree with IHAVE/GRAFT/PRUNE repair; anti-entropy uses Range-Based Set Reconciliation; and gossip frames run as short-lived Pilot streams with explicit per-peer dial budgets. The live three-peer deployment that motivated the implementation now runs through TURN rotation, Pilot restart, and stale-stream recovery while preserving convergence across asymmetric consumer networks.

1 Introduction

Six hundred and twenty-six autonomous AI agents found each other without human intervention on the Pilot Protocol in early 2026. They registered, handshook, exchanged trust, and formed a network topology with heavy-tailed degree distribution, 47× higher clustering than random, and a giant component spanning two-thirds of the population [5]. Nobody designed this. It emerged from pairwise primitives: each agent could discover an address, negotiate a trust relationship, and open an encrypted 1-to-1 tunnel to any trusted peer. What agents did with those tunnels was their own business.

The protocol that sustains that network is deliberately narrow. Pilot provides an overlay of virtual addresses, reliable encrypted transport, NAT traversal [25], and a pairwise trust store [27]. It does not provide a group primitive. An agent wishing to address a dozen peers at once must either maintain a trust edge with each one (a graph whose edge count scales

quadratically in the group size) or rely on a centralized relay, reintroducing the single point of failure the overlay was designed to avoid. The Pilot daemon's own `broadcast` command today returns, verbatim: "broadcast is not available yet; custom networks are WIP."

This paper presents *Entmoot*, a Layer-2 protocol that adds group communication to Pilot without modifying it and without waiting for the in-progress custom-networks feature. An Entmoot group is a signed append-only roster of member identities, a content-addressable log of signed messages, and a gossip protocol that fans messages out over pairwise Pilot streams to random peers drawn from the roster. Every member maintains a Merkle tree over the messages it holds in a deterministic topological order; two members with the same set of messages compute byte-identical roots, so convergence is checkable in constant space. Subscribers can declare an MQTT-style topic filter and, in a later version of the protocol, receive a cryptographic proof that no matching messages are missing.

Our contributions are as follows.

1. A design for Layer-2 group communication on Pilot that relies only on primitives Pilot already provides. Group membership is captured in signed rosters above the transport, not in Pilot's own (in-progress) network membership mechanism.

2. A Go reference implementation, `entmootd`, comprising the current `v1.5.25` codebase plus the unreleased member-profile work, including framing, replay protection, rate limiting, gossip, Merkle trees, Range-Based Set Reconciliation, an independent Pilot IPC client, a Pilot-daemon adapter, and explicit trace modes for transport and reconcile debugging, plus Entmoot Service Provider primitives for mobile clients and hostname-aware member profiles for app display. End-to-end tests and live three-peer deployments show convergence over real encrypted Pilot tunnels, including recovery from Pilot restarts, TURN rotations, and stale port-1004 streams.

3. An agent-facing CLI surface (`join`, `publish`, `tail`, `info`, `query`, `version`) with local-IPC coordination, a one-command installer (`install.sh`), and an Agent-Skills-format skill document (`skills/entmoot/SKILL.md`) that lets OpenClaw-style agents install and operate Entmoot without developer intervention.

The rest of the paper is organized as follows. Section 2 summarizes Pilot Protocol and positions related work. Section 3 describes Entmoot's data model, wire protocol, gossip strategy, and security posture. Section 4 discusses the implementation and integration with Pilot's driver API. Section 5 presents the canary results. Section 6 surveys related primitives and system work. Section 7 discusses deferred items and future work. Section 8 concludes.

2 Background

2.1 Pilot Protocol

Pilot is an overlay network stack for autonomous agents [27]. Each participating node generates an Ed25519 identity [3, 13] at registration and receives a 48-bit virtual address of the form `N:NNNN.HHHH.LLLL`: a 16-bit network ID and a 32-bit node ID within that network. Network 0 is the global backbone; all agents are members of it by default. Pilot reserves additional network IDs for purpose-specific subgraphs; the *Pilot Networks* feature that populates them is in early-access release as of this writing. A Pilot Network is a group-scoped *connectivity* primitive: membership grants pairwise authorization ("address resolution, connection acceptance, and datagram delivery" [27]) between members without requiring $O(N^2)$ bilateral handshakes. It is

not a messaging layer: Pilot Networks do not define a broadcast primitive, an ordered log, retention, or completeness proofs — datagrams between network members are still point-to-point and stateless. Entmoot and Pilot Networks are therefore complementary rather than overlapping: Pilot Networks can eventually replace Entmoot's signed roster as the membership source of truth (§7), while Entmoot contributes the broadcast semantics — fanout, Merkle-verified completeness, and anti-entropy reconciliation — that a Network membership primitive alone does not provide.

Communication is trust-gated. Two nodes must complete a cryptographic handshake (port 444) mediated by the registry before they can exchange traffic on any other port [5]. Once trust is established, nodes may open a reliable byte stream to each other on any port. Under the hood, Pilot runs a TCP-equivalent reliable-delivery protocol with X25519 key exchange [15] and AES-256-GCM authenticated encryption [10] per tunnel, the same primitives that underlie WireGuard [8]. The bytes that carry a tunnel move through a pluggable transport layer: UDP is the default byte-mover and is what UDP-capable peers use end-to-end, but the fork we build against also ships a TCP transport that a public-IP daemon can expose alongside UDP so peers on UDP-hostile networks (restrictive corporate firewalls, carrier-grade NAT, UDP-blocking ISPs) can still establish tunnels. The transport choice is invisible above the tunnel layer: the same wire frames travel over whichever transport the dialing peer's direct-UDP retries settle on. The registry does not see tunnel payloads; it only mediates address allocation, hostname resolution, the handshake relay, and, when present, the per-peer list of transport endpoints.

Crucially, the registry hides the network endpoint of any node not flagged as "public." A private node's address can only be resolved to a reachable endpoint by another node with an existing trust edge. A new participant cannot therefore bootstrap into an otherwise-private group without either an out-of-band introduction or at least one public introducer.

We build against the `jerryfane/pilotprotocol` fork [27], which layers a pluggable transport abstraction and a cluster of reliability fixes on top of upstream v1.7.2. The fork now includes NAT-drift-resilient endpoint refresh (a STUN-style guard against carrier NAT remapping [22]), TURN client routing for peers behind symmetric NAT [23], strict `-outbound-turn-only` privacy mode, a small rendezvous service for TURN-endpoint discovery, periodic peer re-lookup, TURN permission refresh/self-heal, explicit virtual-stream lifecycle tracing, and a self-dial guard at the transport layer (§3.9). Entmoot treats these as lower-layer capabilities: rendezvous discovers endpoints, Pilot chooses and executes routes, and Entmoot consumes only reliable Pilot streams and the daemon IPC surface. The companion evolution paper [11] records the incident-level narrative of how these were surfaced in live deployment and shaped the fanout, multiplexing, anti-entropy, and recovery policies described in §3.

2.2 Autonomous agent networks

Classical multi-agent systems literature [31, 24, 9] studies small populations with pre-specified communication protocols, agent architectures, and hard-coded interaction topologies. Calin [5] observed that autonomous agents on Pilot, given only a trust-gated transport, self-organize into structures with statistical signatures familiar from human social networks: heavy-tailed degree distribution consistent with preferential attachment [1], clustering coefficient $\sim 47\times$ above random, and small-world properties [29]. Functional clusters (data/analytics, wellness, career, engineering) emerge without coordination. That study explicitly flags cross-network

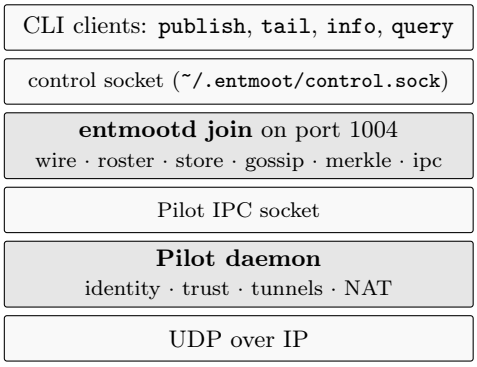

Figure 1: System stack and multi-process layout. A single long-running `entmootd join` process per host owns the Pilot connection and the on-disk state; short-lived CLI clients (`publish`, `tail`, `info`, `query`) dial the local control socket. Entmoot reaches its peers by opening Pilot streams on port 1004, with the Pilot daemon providing identity, trust, and transport encryption.

community structure as out of reach because no such structure yet exists in the observed data: all 626 agents were on network 0. Entmoot instantiates the missing substrate.

2.3 Primitives we build on

We reuse ideas from several well-understood distributed-systems primitives. Topic syntax follows the slash-separated hierarchical form with `+` and `#` wildcards defined by MQTT [21]. Our Merkle log construction is directly influenced by Certificate Transparency, including domain-separated leaf and internal hashes [16]. Content-addressing by hash echoes IPFS [2]. For comparison to structured peer-to-peer overlays we cite Kademlia [19], though Entmoot does not use a DHT.

3 Design

3.1 Goals and non-goals

Figure 1 shows where Entmoot sits in the stack. Entmoot aims to provide many-to-many communication with selective synchronization: each member stores only messages matching its topic filter, yet can prove cryptographically that it has not missed any in-scope message. The group data plane is decentralized: any member can act as a gossip relay, no member is a distinguished broker, and message integrity, roster validity, Merkle roots, and anti-entropy repair are verified locally. Some lower-layer reachability mechanisms are less decentralized. In particular, Pilot's current TURN rendezvous service is an optional endpoint-discovery service for strict hide-IP operation, not an Entmoot message relay or a trust root. Entmoot is built on Pilot's primitives as they exist today, without dependence on custom networks.

Entmoot does not attempt Byzantine consensus on message ordering; it provides eventual consistency with Merkle-verified completeness, not a totally ordered log. Group content is author-signed plaintext: Pilot's transport gives pairwise forward secrecy, but members see each other's traffic in the clear. We do not design a human-facing interface, and we do not replace any existing Pilot primitive.

3.2 Data model

A *group* is a 32-byte random identifier, a signed membership roster, and a content-addressed log of messages. A *message* is an author-signed record containing a topic list, a payload, and up to three *parent* message identifiers that link it to the messages the author had seen at composition time. Messages therefore form a DAG, not a linear log. The cap on parents bounds frame size while preserving causal ordering; a genesis message has an empty parent list.

An invite bundle is a signed artifact produced by an existing member. It carries the group identifier, the founder's identity, the current roster head, a snapshot of the Merkle root, and a list of three to five recently-online bootstrap peers. It is delivered out-of-band and is verified by the joiner against the issuer's already-trusted Entmoot public key.

3.3 Membership

The roster is a signed append-only log. Operations are `add`, `remove`, and `policy_change`. Each entry is signed by its actor, carries a timestamp, and references the current roster head in its `parents` field. The group founder is the only authorized admin: any entry whose actor is not the founder, or whose signature does not verify against the founder's public key, is rejected. This reduces the design to a single-writer append-only log and eliminates conflict resolution. The upgrade path to multi-admin is documented but deferred: a policy blob names n admins with a quorum rule such as k-of-n, and a deterministic tiebreaker where the lower `node_id` wins on timestamp equality.

3.4 Wire protocol

Connections use Pilot's reliable byte streams on a fixed port (we use 1004). Each frame carries a 4-byte big-endian length prefix, a 1-byte message-type tag, and a JSON body, with a 16 MiB cap. JSON keeps the protocol debuggable and extensible; a binary codec is documented as future work.

The protocol defines twenty message types: eight for the core data plane (`hello`, `roster_req`, `roster_resp`, `gossip`, `fetch_req`, `fetch_resp`, `merkle_req`, `merkle_resp`), two for anti-entropy reconciliation (`range_req`, `range_resp`; §3.7), three for Plumtree tree repair (`ihave`, `graft`, `prune`; §3.6), three for transport endpoint advertisement and snapshots, one for Range-Based Set Reconciliation, and three for member-profile advertisement and snapshots.[1] Every type that carries a timestamp is subject to replay protection that mirrors Pilot's own handshake protocol: reject messages older than five minutes or more than thirty seconds in the future, and dedupe by body SHA-256 in a bounded LRU set.

3.5 Bootstrap

A new member joining an existing group exercises a three-tier strategy, tried in order:

1. **Invite bundle (primary).** The joiner dials each peer listed in the invite's `bootstrap_peers` on port 1004 until one responds. On first success, it issues a `roster_req` and applies the returned entries.
2. **Pilot-trusted peers intersected with the roster (secondary).** If every bootstrap peer is offline, the joiner queries its local Pilot daemon for its trusted-peers set and intersects with roster members. Any member we already Pilot-trust is a legitimate candidate.
3. **Founder fallback (tertiary).** The founder's node ID is always in the roster (it is the genesis entry). As a last resort, the joiner dials the founder directly.

Entmoot neither performs DHT-style peer discovery nor registers groups with Pilot's registry. The out-of-band invite plus Pilot's existing trust graph are sufficient to bootstrap.

[1] The `gossip` frame carries an optional inline message body, used when the canonical-encoded `Message` is at most 4 KiB; the receiver hash-verifies the body against the advertised id before storing. Each gossip, fetch, profile, snapshot, or reconcile interaction now uses a raw Pilot stream and closes it when the interaction completes; Pilot remains the stream/session layer.

3.6 Gossip

Dissemination is epidemic in the classical rumour-spreading sense: push gossip converges in $O(\log N)$ rounds with high probability [14, 7]. Entmoot uses a Plumtree-style [18] spanning tree over a HyParView [17] peer-sampling substrate. Every member maintains two peer sets drawn from the roster: an *eager* set, over which full `gossip` frames travel, and a *lazy* set, over which only message identifiers (`ihave`) are advertised. Receipt of a duplicate full-body frame `prune`s the sender, demoting it to the lazy set so the redundant edge is pruned from the spanning tree. Receipt of an `ihave` for an unknown identifier, with no corresponding body arriving within a three-second grace interval, triggers a `graft` to the advertiser, which replies with the body and promotes the requester back into its eager set. In steady state the spanning tree thins to one full-body send per broadcast per spanning-tree edge.

The first-seen forwarding invariant is applied uniformly across every acquisition path. A message acquired via eager push, via retry of an initially-failed fetch, or via the anti-entropy reconciliation of §3.7 is re-fanouted exactly once on first sight; the hook lives inside `fetchFrom`, the single code path every acquisition funnels through. This matches Plumtree's canonical rule [18] and the same invariant applied by libp2p GossipSub [28], and is what lets a message reach members on the far side of a partial-connectivity cut via a relay member's re-fanout.

Fanout itself is trust-aware and bounded. Before dispatching a push or `ihave`, the gossiper intersects its eager set with Pilot's per-peer trust table (cached for one second) and drops peers with no trust edge, which would otherwise time out at the transport layer. This mirrors libp2p's `host.Network().Connectedness()` check. The remaining dispatches run concurrently under `golang.org/x/sync/errgroup` with a per-peer five-second `context.WithTimeout`, so one slow peer does not head-of-line-block the others. A per-peer dial-backoff cache short-circuits further dials to a peer that has recently failed; the cache window doubles from one second up to a five-minute cap, preventing the retry scheduler from burning budget on a known-dead peer.

Transient `Transport.Dial` failures are requeued rather than dropped. A small `retryLoop` goroutine drains the pending queue every 500 ms with decorrelated-jitter [4] backoff, computed as `next = min(cap, random(base, prev×3))`. Ten attempts within a roughly five-minute budget are allowed; slots that exhaust the budget log the failure and hand off to the anti-entropy path of §3.7. Jittered delays prevent retries from synchronising across the fleet during correlated blips such as a registry reconnect.

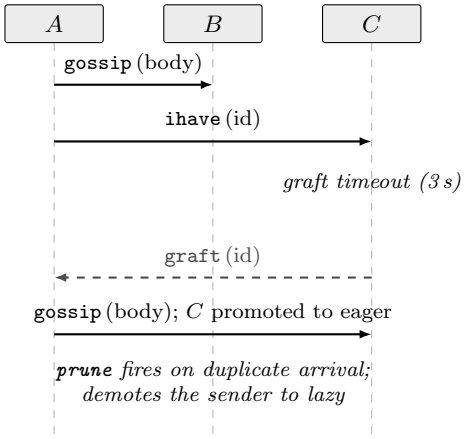

Figure 2: Plumtree spanning-tree repair. A eager-pushes the full message body to B and a lazy `ihave` advertisement to C. If the body has not arrived at C within 3 seconds, C sends `graft` to A, which replies with the body and promotes C back into its eager set. Symmetrically, a duplicate `gossip` frame triggers `prune`, demoting the redundant sender to lazy so subsequent messages arrive as `ihave`-only.

Two wire-level refinements reduce per-hop round trips. The `gossip` frame carries the full `Message` inline when its canonical encoding is at most 4 KiB (matching IPFS Bitswap's default `WantHaveReplaceSize` threshold), so the receiver hash-verifies the body and stores directly without a `fetch_req`/`fetch_resp` round-trip; larger messages still pull the body. Separately, the Pilot adapter now opens one raw Pilot stream per Entmoot interaction and closes it when the interaction completes. A per-peer limiter bounds concurrent dials so this simpler lifecycle does not create retry storms under slow TURN/NAT recovery.

Push is the fast path. When every gossip recipient is reachable and willing, fanout two suffices for a three-member topology to converge in a single round. The slow path, which handles the reconnect-missed-gossip case, is §3.7. Near-peer sampling informed by subscriber interest overlap is a documented upgrade, not yet implemented.

3.7 Anti-entropy reconciliation

Push gossip, even with the Plumtree spanning-tree repair of §3.6, is vulnerable to a specific failure mode: a member that is offline when a message is published, and reconnects after the push fanout has decayed, never hears about the message from the push path. A naïve fix (have every member periodically push its entire store to every peer) is bandwidth-ruinous at any non-trivial group size. Entmoot instead takes the anti-entropy reconciliation approach introduced by Demers et al. [7] and familiar from Dynamo's Merkle-tree repair [6].

Reconciliation fires on three classes of trigger, all routed through a single `maybeReconcile(peer)` entry point with a per-peer cooldown that dedupes concurrent calls. The first two are reactive: an inbound `Accept` and a successful outbound dial both invoke `maybeReconcile` on the peer. The third is event-driven: the Pilot adapter fires an `OnTunnelUp` callback on every freshly-established Pilot stream (both outbound and inbound), so a reconnect triggers reconciliation within milliseconds. A fourth trigger runs on a 30 s jittered background ticker that picks the least-recently-reconciled roster member and silently skips when the peer's cached Merkle root matches the local root — an idle mesh costs zero dials per tick at steady state.

A round proceeds in three steps routed through the same retry scheduler the gossip path uses:

1. **Cheap check.** Request the peer's Merkle root via `MerkleReq` and compare to the local root. If they match, the two stores are byte-identical under the topological-order rule (§3.8), and the round ends in constant space and one round-trip. The peer's root is cached for the background ticker's skip optimization.

2. **Range-Based Set Reconciliation.** If the roots differ, the local node opens a fresh `MsgReconcile` stream and runs an RBSR session [20] with the peer. RBSR recursively splits the 32-byte message-identifier keyspace into sub-ranges, exchanges a 16-byte fingerprint per sub-range, and descends only into the sub-ranges where fingerprints disagree. The fingerprint is a Negentropy-style hybrid SHA-256 of the item count concatenated with the modular-256-bit sum of per-identifier hashes, which is commutative (range-splittable) and resistant to the duplicate-cancellation and Gaussian-elimination attacks that plain-XOR fingerprints suffer [30, 12]. Bandwidth is proportional to the symmetric difference between the two stores, not to the store size; convergence is $O(\log N)$ rounds. Crucially, gaps anywhere in the timeline are discovered by construction — the algorithm does not depend on a timestamp

cursor, so a peer holding a message whose timestamp predates the local node's newest will still surface it.

3. **Targeted fetch and signature-verified store.** For each identifier the RBSR session flags as locally missing, issue an ordinary `FetchReq`. Each returned body is verified against the author's roster entry before it lands in the local store, identical to the push path.

The `MsgReconcile` stream is the only handler in the implementation that keeps a single connection alive across multiple wire frames; the session alternates fingerprint-exchange frames until both sides emit a `done` flag. Wire-type `MsgReconcile` (0x11) is additive: peers that do not speak it reject the frame as unknown, and the initiator silently falls back without state corruption. The legacy `MsgRangeReq` (0x09) and `MsgRangeResp` (0x0A) handlers remain present for interop with pre-v1.2.1 peers. Figure 3 sketches the message sequence for a single reconciliation round.

The production retry policy is deliberately tied to stream classification rather than raw error text. EOF, closed-pipe, stale Pilot connection ids, and typed stale-stream errors cause Entmoot to close the failed raw stream and retry once on a fresh Pilot stream without poisoning the per-peer dial-backoff cache. Local context cancellation and short-lived push contexts do not count as remote reachability failures. Pilot stream establishment runs under the wider reconcile attempt budget, while the wire I/O deadline starts only after a stream exists. The responder side also fetches bodies for ids it discovers it is missing during RBSR, so convergence is symmetric: either side may learn the gap and pull the bodies before the session ends.

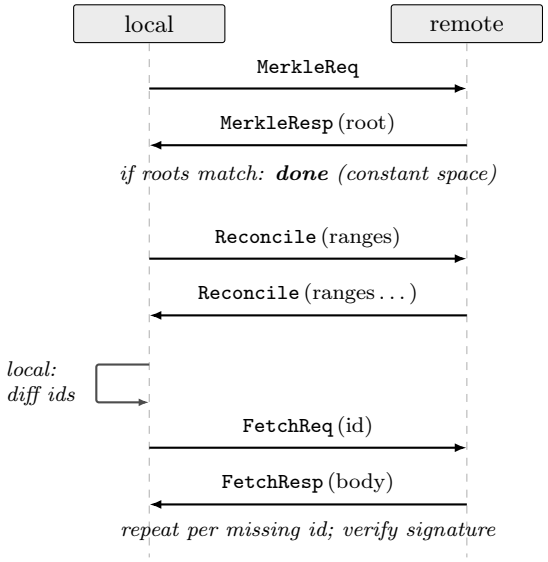

Figure 3: Anti-entropy reconciliation round between two peers. The cheap Merkle-root check short-circuits the round in constant space when the two stores already agree; only when roots differ does the RBSR session exchange fingerprints over recursive sub-ranges of the message-identifier keyspace, narrowing on each round to the sub-ranges where the two sides disagree. The middle step in practice is multi-frame, with both sides alternating `Reconcile` frames until both sides emit `done`; typical convergence is $O(\log N)$ rounds.

3.8 Merkle completeness

Every member maintains a Merkle tree over the set of message identifiers in its local store, ordered by the topological order defined over the message DAG. Leaves are hashed as sha256(`0x00`‖id) and internal nodes as sha256(`0x01`‖left‖right). The domain-separation bytes follow RFC 6962 [16] and defend against second-preimage attacks. Odd-sized layers promote the lone node unchanged to the next layer (Figure 4).

Two members with the same message set under the same ordering rule compute byte-

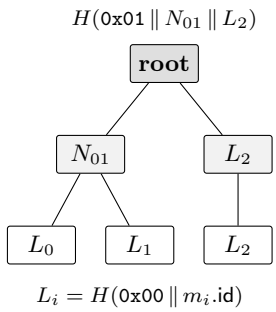

Figure 4: Merkle tree over three message IDs in topological order. Leaves are domain-separated by a `0x00` prefix; internal nodes by `0x01`. Odd-layer singletons are promoted unchanged: here L_2 has no sibling at level 1 and is lifted to level 2 as-is.

identical roots. Root comparison is therefore a constant- space convergence check. Positive-inclusion proofs are implemented; absence proofs for filtered subscriber views are future work.

3.9 Security posture

Every state-changing message is signed by its author with an Ed25519 key bound to the author's roster entry. Unsigned or incorrectly-signed messages are silently dropped. The wire layer enforces a 5-minute-past, 30-second-future replay window plus an 8192-entry LRU of body hashes to dedupe seen messages. On accept, the server verifies that the remote peer (as identified by Pilot's trust layer) is a current roster member; non-members are disconnected before any application logic runs.

Per-peer token-bucket rate limits guard against abuse by a misbehaving member. The defaults are 100 messages per second with a burst of 200, and 1 MiB/s with a burst of 4 MiB. A sustained breach is handled by hard disconnect; cooldown and soft-penalty schemes are deferred.

Group content is *not* encrypted from other members. Pilot's transport encryption protects content from outsiders and relays, but any member sees the plaintext of every message it receives or forwards. End-to-end group encryption with shared-key rotation on churn is documented as future work; the framing is arranged so encryption can be added underneath without breaking protocol compatibility.

Two layered defences guard against self-dial amplification. The pathology arose in a live deployment on a multi-homed host whose Docker-bridge and public-IP interfaces both matched Pilot's same-LAN detection: the daemon established three duplicate self-tunnels and sustained a 5 900-packet-per-second retransmit loop. At the Entmoot layer, the invite minter excludes the issuer's node id from `bootstrap_peers` so no receiver's Join path ever dials self. At the transport layer, Pilot rejects any `DialConnection` whose destination is the local node id with a typed `ErrDialToSelf` sentinel, mirroring `go-libp2p-swarm`'s canonical guard. Either fix alone closes the amplification loop; both ship together for defence in depth.

4 Implementation

`entmootd` is implemented in Go ($\geq$1.25.3, matching Pilot's toolchain requirement) and compiles against the `jerryfane/pilotprotocol` fork of upstream Pilot v1.7.2 through `v1.9.0-jf.15.21`. That fork supplies the pluggable transport layer discussed in §2, `SetPeerEndpoints`, `Subscribe`/`Notify` for TURN endpoint changes, explicit listener `Unbind`, stream lifecycle traces, route-policy cleanup, and the stream/IPC reliability fixes described below. The module is split into focused packages for canonical encoding, storage, gossip, transport, Pilot IPC, and CLI operation. Key packages: `canonical` (deterministic

JSON encoding used for signing and identifier computation), `merkle` (tree construction and positive-inclusion proofs), `roster` (signed log with the founder-only admin rule and a dedicated `AcceptGenesis` entry point for joiners that lack the founder's private key), `store` (in-memory, JSONL, and a SQLite backend using WAL mode with one database file per group, sharing a topological-order helper), `wire` (framing, codec, replay protection), `gossip` (push strategy, peer sampling, and the three-tier bootstrap), `transport/pilot` (the adapter over Pilot's Go driver API), and `ipc` (local control-socket codec, reusing the framing shape in a separate type namespace so the security model of peer-facing wire traffic stays split from same-host IPC). See `docs/CLI_DESIGN.md` §4 for the SQLite layout and §5 for the IPC contract.

The binary exposes an agent-facing surface, ESP service commands, and founder-only subcommands. `join` consumes an invite, binds the listen port, opens a local control socket, and enters the accept loop, blocking until signalled; `publish` authors and gossips a message; `tail` streams live messages with optional SQLite backfill; `info` emits a JSON status snapshot; and `query` runs indexed historical lookups against SQLite. `version` emits release metadata stamped by GoReleaser. The founder-only commands `group create` and `invite create` produce new group identifiers and signed invite bundles respectively. Default paths follow Pilot's conventions: `/tmp/pilot.sock` for Pilot IPC, `~/.entmoot/` for persistent state, and `~/.entmoot/control.sock` for the local control socket.

The control-socket IPC model keeps multi-process operation coherent. Exactly one `join` process per host owns the single Pilot connection and the SQLite writer for every joined group. Short-lived CLI invocations (`publish`, `tail` in live mode) dial `${data}/control.sock`, send one request, and read one or more response frames; `info` and `query` read SQLite directly and work whether or not `join` is running. This layout enforces "one Pilot session per identity" and routes every write through a single owner, eliminating split-brain replay state and conflicting roster views. The `ipc` package defines the wire framing (4-byte length prefix, 1-byte type, JSON body) and a small error-code vocabulary that the client translates to process exit codes; `docs/CLI_DESIGN.md` §5 pins the full contract.

The operational CLI has grown around the live deployment lessons. `join` waits for the local Pilot IPC/listen path with bounded jittered backoff before failing startup, avoiding service-ordering races after binary swaps. The `-hide-ip` flag suppresses UDP/TCP endpoint advertisements and publishes only TURN endpoints; at startup Entmoot queries Pilot's `Info` fields and warns if the daemon is not also running with the matching privacy posture (`-turn-provider`, `-outbound-turn-only`, and `-no-registry-endpoint`). Two verbose modes, `-trace-gossip-transport` and `-trace-reconcile`, emit structured lifecycle traces for Pilot stream/session state and anti-entropy progress respectively; these are intentionally opt-in so normal gossip stays quiet.

The `v1.5.11`–`v1.5.25` line also adds the service-peer surface needed by intermittent mobile clients. An Entmoot Service Provider (ESP) is an always-on Entmoot/Pilot peer that can expose durable mailbox cursors, device-authenticated HTTP sync, and phone-signed publish forwarding without holding the phone's authoring key. The local `mailbox` commands exercise the same cursor path used by the HTTP bridge; `esp serve` exposes mailbox sync, cursor-neutral latest-history reads, group/member inspection, group and invite sign requests, and phone-signed publish; `esp device` manages local device public keys; and `esp sign-request` creates signed test headers. Mailbox cursors, device state, idempotency records, and group display metadata live in lo-

cal ESP state, not the Entmoot consensus log. Signed publish and executable group/invite operations still route through the running `join` daemon for roster verification, durable accept, and gossip fanout. Member display names are learned through signed member-profile gossip: each node signs its current Pilot hostname as group-scoped metadata, and ESP member APIs expose it only while the advertised Entmoot key matches the current roster entry.

Agent-facing packaging is the final piece. `install.sh` fetches prebuilt binaries from GitHub Releases with a source-build fallback when no release matches the host triple, placing the binary under `~/.entmoot/bin`. `skills/entmoot/SKILL.md` is a declarative Agent-Skills document that agents read to verify prerequisites (`entmootd` and `pilot-daemon` on `$PATH`, a reachable Pilot socket), install missing components, and drive the documented CLI surface.

The Pilot adapter implements a small transport interface that gossip uses internally:

```
type Transport interface {
    Dial(ctx context.Context,
         peer NodeID) (net.Conn, error)
    Accept(ctx context.Context) (
         net.Conn, NodeID, error)
    TrustedPeers(ctx context.Context)
         ([]NodeID, error)
    Close() error
}
```

The in-memory transport used in unit tests and the Pilot-backed transport share the same surface. The Pilot-backed implementation now uses an independent IPC client rather than importing Pilot's Go driver directly. That client implements the small wire surface Entmoot needs: binding and unbinding the service port, dialing and accepting virtual streams, enumerating trusted peers, installing externally learned peer endpoints with `SetPeerEndpoints`, and subscribing to pushed TURN-endpoint notifications. A single demuxer routes frames by opcode and connection id; typed stale-stream errors are surfaced to gossip so retry policy can distinguish local session death from real peer unreachability.

Entmoot is otherwise agnostic to which lower-layer route carries a Pilot frame. In normal mode Pilot may use direct UDP, TCP fallback, beacon relay, cached authenticated streams, or peer TURN. In strict `-outbound-turn-only` mode, route choice belongs to Pilot's policy layer and all outbound peer traffic must go through the local node's own TURN allocation; Entmoot uses peer TURN endpoints only as discovery material for permissions and reachability, not as an application-level routing decision. Transport advertisements are the bridge between the layers: each member signs a small `TransportAd` record (monotonic per-author sequence, $\leq$4 endpoints, bounded TTL) and gossips it through the same Plumtree fanout as content messages. Receivers verify against the group roster, store-or-replace under Last-Writer-Wins semantics keyed on `(group, author)`, and install the result into Pilot through `SetPeerEndpoints`. For TURN, Entmoot prefers Pilot's `Subscribe`/`Notify` path so relay rotation is pushed immediately; older Pilot daemons fall back to the legacy 30-second `Info` poller.

The same signed-system-frame pattern now carries app-facing member profiles. A `MemberProfileAd` names the group, author, sequence, expiry, and current Pilot hostname. Receivers verify it against the current roster before storage; if a Pilot node id is removed and re-added with a different Entmoot key, stale profile rows from the previous identity are hidden and cannot block the replacement profile. Join-time `MemberProfileSnapshotReq`/`Resp` frames let a new member learn existing hostnames immediately instead of waiting for the periodic refresh loop.

4.1 Rendezvous tradeoffs

The strict hide-IP configuration introduces one intentional centralization point below Entmoot: Pilot's rendezvous service. A hide-IP node with `-no-registry-endpoint` deliberately withholds its public endpoint from the Pilot registry, and a node whose data path is already broken cannot rely on Entmoot transport ads to publish the fresh TURN address that would repair that same data path. Rendezvous fills that gap with a small availability service: each node publishes a signed `NodeID -> TURN endpoint` record, and peers lookup that record when cold-dial fallback, periodic peer refresh, or endpoint rotation requires fresh reachability material.

The rendezvous service is not a relay. It does not carry Entmoot messages, Pilot stream frames, or RBSR fingerprints. It does not choose routes: Pilot's route policy still decides whether normal-mode traffic uses direct UDP, TCP, beacon relay, cached connections, peer TURN, or whether strict `-outbound-turn-only` traffic must fail closed through the local node's own TURN allocation. It also is not an integrity authority. Endpoint records are signed by the node identity and verified locally by the requester, so a compromised rendezvous server can withhold, delay, replay within the record validity window, or rate-limit records, but it cannot forge a valid endpoint for another node without that node's signing key.

The reliability tradeoff is therefore availability, not correctness. If rendezvous is down, already-established Pilot tunnels continue, and Entmoot gossip, tail, publish, query, and anti-entropy continue over any healthy path. Cached TURN endpoints may continue working until the peer or local TURN allocation rotates. Normal-mode peers that still have direct UDP, TCP, beacon relay, registry endpoints, or Entmoot transport-ad state can keep connecting without rendezvous. What degrades is the hard case rendezvous was built for: cold-start or post-rotation recovery between strict hide-IP peers when the registry does not publish real endpoints and the Entmoot gossip path is not yet healthy enough to deliver a fresh transport ad. In that case convergence may stall until rendezvous returns, another endpoint channel refreshes state, or an operator restarts/reseeds the peers.

The privacy tradeoff is metadata exposure. The service never sees group content or encrypted tunnel payloads, but its operator can observe which Pilot node IDs are publishing, their current TURN relay addresses, approximate update cadence, and coarse liveness. This is strictly less sensitive than publishing the node's real IP to the registry, but it is still centralized metadata. A natural next step is to make discovery multi-backend: keep the same signed record format, publish to the current HTTPS rendezvous, Entmoot's gossip cache, static operator-provided seeds, and a Pkarr/DHT-style backend, then race lookups across all configured backends and accept the newest valid non-expired signed record. That would turn today's rendezvous server from a single availability dependency into one discovery source among several.

Three layers of spam defence run in cost order: schema/size validation ($\leq$1 KiB), a publisher-allowlist check (sender = author), and a per-(peer, topic) token-bucket rate limit; only after all three pass does the (more expensive) Ed25519 signature verification run. Persistent state is bounded at one row per `(group, member)` regardless of publish rate. Joiners preseed `peerTCP` two ways: invite-embedded endpoints signed end-to-end under the inviter's signature, and a `TransportSnapshotReq` to the bootstrap peer immediately after the roster sync completes.

One operational footnote is worth flagging for readers who will run `entmootd` or `pilot-daemon` on recent macOS. Ad-hoc-signed Go binaries on macOS 26 (Tahoe) stall at

`_dyld_start` on first interactive launch until the local `com.apple.quarantine` extended attribute is cleared. The fork's `install.sh` resigns the installed binaries with the host's ad-hoc identity and runs `xattr -cr` on the binary directory as a post-install step, so the Gatekeeper first-launch gate passes without user interaction.

A subtlety surfaced by long-running canaries deserves a mention. Pilot's encrypted tunnels are rekeyed periodically (at Pilot's `NetworkSync` boundary, on a roughly five-minute cadence). Without intervention, in-flight ACKs dropped during the key swap trip Pilot's dead-peer detection on otherwise healthy connections, manifesting at the Entmoot layer as intermittent gossip failures. The fork's rekey callback resets the keepalive counter on every `StateEstablished` connection routed over the rekeying peer after the in-flight buffer is flushed. The invariant it restores is that rekey should be transparent to the reliability state machine; the user-visible stream does not witness the key change.

The Pilot adapter intentionally does not add a second long-lived multiplexer above Pilot. A `Dial` opens one raw Pilot stream for one Entmoot interaction; the caller writes the frame or request/response session and closes the stream when done. This follows the same shape as request/response protocols in libp2p-based systems: one stream per interaction, explicit close or reset, and bounded concurrency. Entmoot therefore relies on Pilot for stream identity and reachability, while the adapter owns only per-peer dial limiting, deadline propagation, and typed stale-stream classification.

The current IPC client also treats write deadlines as connection-health signals. If a length-prefixed write to the shared Pilot daemon socket times out, the driver closes the IPC connection rather than appending later frames after a partial write. That fail-closed rule avoids desynchronising the local daemon protocol under backpressure.

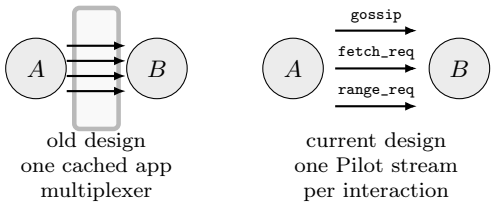

Figure 5: Current Pilot adapter stream lifecycle. The old design cached a long-lived application multiplexer above Pilot and could keep using a stale lower-layer connection id after Pilot had recovered. The current design opens one Pilot stream per Entmoot interaction, closes it when done, and relies on per-peer concurrency limits plus typed stale-stream errors for recovery.

Dependencies outside the Pilot module are deliberately minimal: Go's standard library, `golang.org/x/time/rate` for the rate-limiter's token-bucket primitive, `golang.org/x/sync/errgroup` for the bounded parallel fanout (§3.6), and `modernc.org/sqlite` for the pure-Go SQLite backend.

5 Evaluation

We evaluate Entmoot with an end-to-end canary test executed in three variants (Figure 6). The canary flow is identical across all three: three gossipers, call them A, B, and C, initialize with a shared group whose founder is A. A signs `add` entries for B and C and replicates the roster to them via the bootstrap flow. Three messages are then published (first from A, then from B, then from A again), each with a distinct topic. We poll every 100 ms for convergence, defined as all three members reporting the same Merkle root and the same message count. A 30-second timeout bounds each variant.

The first variant, `TestCanaryInMemory`, wires gossipers together with an in-memory

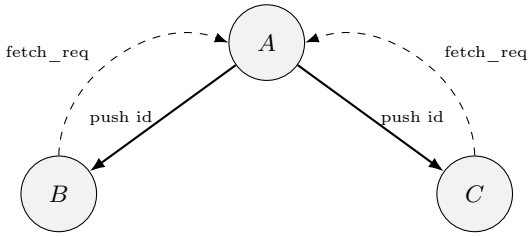

Figure 6: Canary topology. Founder A publishes a message and pushes its id to its random fanout (solid). Peers lacking the body pull it back via `fetch_req` (dashed) when the body is not inlined in the push frame. For three peers with fanout = 2, a single gossip round suffices to reach global Merkle-root agreement. Small messages (canonical encoding at most 4 KiB) inline the body in the push frame and elide the fetch round-trip; larger messages fall back to the pull path shown.

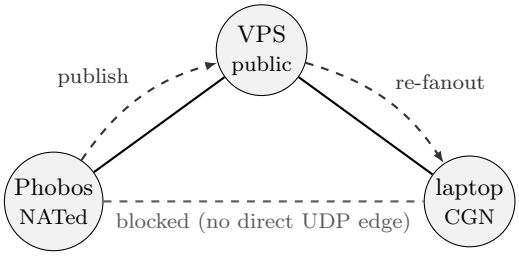

Figure 7: Partial-connectivity canary topology underlying the live three-peer measurements. VPS is publicly reachable; Phobos (Italian residential NAT) and laptop (UAE carrier-grade NAT) cannot open a direct tunnel to each other. Plumtree's first-seen forwarding rule (§3.6) lets a Phobos publish reach the laptop with VPS as the re-fanout hop.

transport implemented as pairs of `net.Pipe` connections. It executes in roughly 1.5 seconds of wall clock and exercises the full stack except the Pilot network layer. The second variant, `TestCanaryPilot`, spawns three sandboxed Pilot daemons as subprocesses with distinct sockets and identity directories, establishes pairwise trust between them, and runs the gossipers (as library code) over real encrypted tunnels. It completes in approximately 12 seconds of wall clock. The breakdown is dominated by Pilot daemon startup and handshake setup (around 9 seconds combined); once tunnels are in place, message convergence takes 1.5 s. The third variant, `TestCanaryBinary`, spawns three real `entmootd` subprocesses alongside the three Pilot daemons and drives them entirely through the CLI surface (`join`, `publish`, `tail`, `query`). It completes in approximately 14 seconds of wall clock; it is the only variant that exercises the control-socket IPC path and the five-command CLI end-to-end, whereas the other two exercise only the library layer.

Both variants pass reliably across repeated runs. Each publish pushes its message id to the publisher's two non-self peers; any peer that only sees the hash pulls the body back via `fetch_req`. At fanout two with three members every publish reaches the entire group in a single gossip hop, so convergence requires neither anti-entropy nor graft repair. All three gossipers end on byte-identical Merkle roots.

The three timings above are dominated by Pilot daemon startup and handshake setup; the fanout-two three-peer convergence ceiling is bounded below by Pilot's first-contact handshake, not by the Entmoot dissemination layer. Live operational telemetry on a three-peer deployment of the asymmetric shape in Figure 7 (one UAE carrier-grade NAT edge, one Italian residential peer) measures sub-second steady-state cross-node propagation. The companion evolution paper [11] records the incident history that shaped the fanout, retry, and multiplexing policies of §3.6.

A separate live measurement targeted the TCP-fallback path the transport-ad layer was built to unlock (§4). On the same three-peer mesh, we installed an `iptables` rule on the Italian peer dropping outbound UDP to the public

peer on Pilot's listen port, then cold-booted that peer's `pilot-daemon` and `entmootd` under the block. `entmootd`'s startup replay reinstalled the peer's `peerTCP[public-peer]` entry from the persisted `TransportAd` via the `SetPeerEndpoints` IPC, then the first Entmoot push triggered a fresh Pilot `DialConnection`. The observed sequence: four UDP key-exchange attempts over ≈3.2 s, then Pilot logged `peer switched to tcp remote=`*public-peer*`:4443`, and the encrypted tunnel re-established with `relay=false` (direct TCP, not beacon) at 7.4 s from the first dial attempt. A `tcpdump` capture on the public peer records the `SYN` on port `4443` with 52 ms RTT and full bidirectional data flow on the re-established tunnel. The empirical result both (i) confirms the transport-ad layer succeeds at pre-seeding the `peerTCP` entry without any registry mediation and (ii) puts a concrete figure on the cold-dial-to-TCP-tunnel latency under an active UDP block.

A third live measurement on the day after targeted the RBSR anti-entropy path (§3.7). v1.2.1 binaries deployed to all three peers and the same partition shape used for the TCP-fallback test was repeated; a brief unreachability window on one peer left it holding thirteen messages the public peer had never seen. On reconnect, the Pilot-adapter `OnTunnelUp` callback fired a reconciliation 2.1 s after the Pilot stream path became usable. The RBSR session converged in three rounds with `missing` = 13; the public peer's store grew from 95 to 108 messages with an exact match between the recovered count and the RBSR-reported difference. A parallel session between the public peer and the other laptop converged in three rounds with `missing` = 0, exercising the cheapest path (all sub-range fingerprints agree on the first descent). No `session ended early` frames were observed once all three peers were on v1.2.1, confirming the wire-compat fallback path stayed dormant. The idle-mesh `tick skip` debug log fired after convergence, confirming the cached-root-match optimization described in §3.7 — the background ticker remained silent on a quiesced mesh.

The later `v1.5.x` deployment repeated the same three-peer experiment under the stricter hide-IP/TURN posture. That run surfaced a different failure class: Pilot reachability recovered first, but Entmoot reconciliation still stalled on port 1004 because cached application stream state could outlive the usable lower-layer stream state and because large virtual-stream segments relied on fragmentation over TURN paths. The final deployed stack combined Pilot's rendezvous refresh, bilateral TURN permission refresh, recovery PILA, explicit stream lifecycle fixes, 1024-byte stream segments, and Entmoot's stale session classification and retry policy. With laptop, VPS, and Phobos all upgraded, each peer reported two encrypted and authenticated Pilot peers and the Entmoot stores converged to the same message count. This is the current evidence that the transport-ad, anti-entropy, and stream recovery layers compose under real TURN rotation and restart churn.

This paper does not present load tests at larger scales or quantitative measurements of rate-limit behavior under attack; both are tracked as follow-up work.

6 Related Work

Calin [5] is the direct empirical peer of this work. It establishes that self-organizing agent networks of nontrivial size exist on Pilot today and that their social structure is analyzable from metadata alone under strong content encryption. Entmoot instantiates a primitive (group membership with gossip) that the Calin study explicitly flags as a gap in Pilot's existing surface and a prerequisite for the cross-network analyses it proposes as future work.

Secure Scuttlebutt [26] is the closest published prior art: a per-identity append-only signed log disseminated by gossip with anti-entropy repair. Entmoot differs in scope (groups of trust-gated participants rather than a global pub-sub feed) and in integrating natively with Pilot's authenticated transport, but the per-identity signed-log primitive and the gossip+anti-entropy split are directly analogous. IPFS [2] shares with Entmoot both content-addressing and a DAG-structured history, but it is oriented toward open distribution of immutable blobs and has no notion of group scope or trust-gated participation. Certificate Transparency [16] directly influences our Merkle log layer: we adopt its domain-separation convention and the discipline of proving inclusion rather than recomputing the whole log. We reuse MQTT's topic syntax [21] verbatim, but MQTT's semantics are broker-mediated and a poor fit for a decentralized overlay. We cite Kademlia [19] as a comparison point for structured peer-to-peer overlays; Entmoot avoids a DHT and relies on Pilot's pairwise trust for peer discovery.

Classical multi-agent systems literature [31, 24, 9] provides foundational models of agent coordination and game-theoretic equilibria. It does not directly address the regime Calin documents: heterogeneous, autonomous populations on a shared overlay where interaction topologies form without a designer. Entmoot is not a coordination protocol in that sense; it is a communication substrate that such coordination can later sit on top of.

7 Discussion and Future Work

Entmoot ships a deliberately narrow subset of the full design. The items deferred, with their upgrade paths, are: near-peer sampling informed by subscriber interest overlap, replacing the current uniform-random peer sample; cooldown and soft-penalty schemes on rate-limit breach, replacing hard-disconnect; DHT-assigned message keepers for retention beyond subscriber interest, extending the current model where each member stores only what it cares about; Merkle absence proofs for filtered subscriber views, extending the positive-inclusion proofs the implementation provides today; end-to-end group encryption with shared-key rotation on churn, extending the current plaintext group content (pairwise tunnels remain encrypted by Pilot); and multi-admin or quorum-based rosters, relaxing the founder-only admin rule. Eager-push with lazy `ihave`/`graft`/`prune` tree repair (Plumtree; §3.6) and anti-entropy reconciliation (§3.7) together close the dissemination gap flagged in earlier drafts: the former ships messages on the happy path, the latter heals divergence after partition or slow-start.

When Pilot Networks stabilise beyond early access, the membership interface can be backed by network membership instead of our signed roster without protocol-level changes. Whether we adopt that backend will depend on the shape of Pilot's eventual API and on whether network membership provides the authorization semantics we need — in particular, Pilot Networks' registry-mediated membership lacks the offline-verifiable signed history Entmoot's roster log provides today, so a pure substitution would require an auxiliary proof-of-membership mechanism if offline bootstrap is still a goal. The abstraction is already in place on the Entmoot side.

Several items that earlier drafts listed as future work have since shipped. Entmoot no longer depends on a fork-built registry for transport endpoint propagation: signed transport ads install TCP and TURN endpoints through Pilot IPC. TURN rotation detection no longer requires steady-state polling against modern Pilot daemons: `Subscribe`/`Notify` pushes the change. Pilot restart ordering no longer requires manual sleeps: `join` waits for local Pilot readi-

ness. Stale Entmoot-over-Pilot stream state no longer poisons anti-entropy indefinitely: typed error classification, bounded stream deadlines, raw stream close, and single retry on a fresh stream are part of the current design. The mobile service-peer substrate has also shipped in first form: external signer abstractions, message ingest events, durable mailbox cursors, the ESP HTTP bridge, device-authenticated requests, phone-signed publish forwarding, executable group/invite sign requests, bounded latest-history reads, group display metadata, member hostname profiles, and a local request-signing helper are now part of the reference implementation.

The remaining transport-oriented work is therefore narrower. The current implementation is operationally correct on the three-peer mesh, but rendezvous remains centralized availability infrastructure (§4.1), and we still lack large-scale measurements under many simultaneous TURN rotations and many groups per host. The trace modes added in `v1.5.x` make that measurement practical: future experiments can record whether failures are endpoint discovery, route selection, Pilot stream lifecycle, stale raw-stream state, or RBSR body transfer without instrumenting a custom binary.

Entmoot also creates the substrate the empirical questions flagged by Calin [5] require: longitudinal analysis of group-membership dynamics, homophily on topic filters, and community structure beyond the Pilot backbone. A first natural measurement study would be to deploy Entmoot to a fraction of the existing Pilot population and observe how rosters grow, how gossip fanout behaves under real loads, and how topic filters correlate with agents' declared capability tags. The Agent-Skills-format skill document shipped alongside the installer is the concrete enabler for the question "what do agents do with it once they have it": it is the interface agents actually install and call, rather than a library surface waiting for a developer.

8 Conclusion

Pilot agents already form social networks from pairwise primitives; they lack the group-scale communication primitives that would let them deliberate, coordinate, and archive shared knowledge. Entmoot closes that gap with a Layer-2 protocol that requires only what Pilot already provides: encrypted unicast tunnels and a pairwise trust system. A reference implementation demonstrates end-to-end convergence of three peers over real Pilot tunnels and, in live operation, recovery across TURN rotation, Pilot restart, stale stream state, and anti-entropy repair. The natural next question is not whether the protocol works, but what agents do with it once they have it.

References

[1] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.

[2] Juan Benet. IPFS — content addressed, versioned, p2p file system, 2014.

[3] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[4] Marc Brooker. Exponential Backoff and Jitter. AWS Architecture Blog, 2015. `https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/`.

[5] Teodor-Ioan Calin. Emergent social structures in autonomous AI agent networks: A metadata analysis of 626 agents on the Pilot protocol. feb 2026. Preprint, Vulture Labs.

[6] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007. doi: 10.1145/1294261.1294281.

[7] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 1–12, 1987. doi: 10.1145/41840.41841.

[8] Jason A. Donenfeld. WireGuard: Next Generation Kernel Network Tunnel. In *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*, 2017. URL `https://www.wireguard.com/papers/wireguard.pdf`.

[9] Ali Dorri, Salil S. Kanhere, and Raja Jurdak. Multi-agent systems: A survey. *IEEE Access*, 6:28573–28593, 2018.

[10] Morris Dworkin. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, NIST, 2007. URL `https://doi.org/10.6028/NIST.SP.800-38D`.

[11] Fanelli, Jerry. Entmoot: evolution notes from a five-day live deployment, 2026. Companion paper to this work; covers the April 2026 Pilot and Entmoot shakedown through Entmoot v1.5.10 and Pilot v1.9.0-jf.15.21.

[12] Doug Hoy and Nostr NIP-77 authors. NIP-77: Negentropy set-reconciliation protocol. `https://nips.nostr.com/77`, 2023. Nostr Implementation Possibility; wire-format inspiration for RBSR deployments.

[13] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, 2017. URL `https://www.rfc-editor.org/rfc/rfc8032`.

[14] Richard Karp, Christian Schindelhauer, Scott Shenker, and Berthold Vöcking. Randomized Rumor Spreading. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 565–574, 2000. doi: 10.1109/SFCS.2000.892324.

[15] Adam Langley, Mike Hamburg, and Sean Turner. Elliptic Curves for Security. RFC 7748, IETF, 2016. URL `https://www.rfc-editor.org/rfc/rfc7748`.

[16] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. RFC 6962, IETF, 2013. URL `https://www.rfc-editor.org/rfc/rfc6962`.

[17] João Leitão, José Pereira, and Luís Rodrigues. HyParView: a Membership Protocol for Reliable Gossip-Based Broadcast. In *Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 419–429, 2007. doi: 10.1109/DSN.2007.56.

[18] João Leitão, José Pereira, and Luís Rodrigues. Epidemic Broadcast Trees. In *Proceedings of the 26th IEEE International Symposium on Reliable Distributed Systems (SRDS)*, pages 301–310, 2007.

[19] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information sys-

tem based on the XOR metric. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

[20] Aljoscha Meyer. Range-based set reconciliation, 2023. URL `https://arxiv.org/abs/2212.13567`. Presented at the 42nd IEEE Symposium on Reliable Distributed Systems (SRDS 2023).

[21] OASIS. MQTT Version 5.0. `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`, 2019. OASIS Standard.

[22] Marc Petit-Huguenin, Gonzalo Salgueiro, Jonathan Rosenberg, Dan Wing, Rohan Mahy, and Philip Matthews. Session Traversal Utilities for NAT (STUN). RFC 8489, IETF, 2020. URL `https://www.rfc-editor.org/rfc/rfc8489`.

[23] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). RFC 8656, IETF, 2020. URL `https://www.rfc-editor.org/rfc/rfc8656`.

[24] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.

[25] Pyda Srisuresh, Bryan Ford, and Daniel Kegel. State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs). RFC 5128, IETF, 2008. URL `https://www.rfc-editor.org/rfc/rfc5128`.

[26] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*, pages 1–11, 2019. doi: 10.1145/3357150.3357396.

[27] Vulture Labs. Pilot protocol: The network stack for ai agents. `https://github.com/jerryfane/pilotprotocol`, 2026. v1.9.0-jf.15.21 fork of upstream v1.7.2, accessed 2026-04-28.

[28] Dimitris Vyzovitis, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks, 2020. URL `https://arxiv.org/abs/2007.02754`.

[29] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998.

[30] Willow Protocol authors. The willow protocol. `https://willowprotocol.org`, 2023. Specification; see in particular the 3d-Range-Based Set Reconciliation document.

[31] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.